

4 – Linear models

Machine Learning (MPA-MLR)

Tomáš Vičar

2025

Department of Biomedical Engineering,
FEEC, Brno University of Technology

Linear models: regression vs. classification

Our general linear equation $\mathbf{z} = W^\top \mathbf{x}$ (last lecture) if we want predict just one value:

$$g(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d$$

Bias trick (augmenting features): We can write an **affine** function as a purely linear one by extending $\mathbf{x}$ (we will use syntax $w/$ or w/o b interchangeably):

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b, \quad \tilde{\mathbf{x}} = \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix}, \quad \tilde{\mathbf{w}} = \begin{bmatrix} \mathbf{w} \\ b \end{bmatrix} \Rightarrow f(\mathbf{x}) = \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}$$

- **Linear regression:** continuous prediction

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b = \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}} \in \mathbb{R}$$

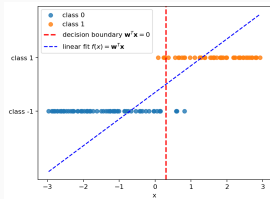
- **Linear classification (2 classes):** class prediction

$$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x} + b) = \text{sign}(\tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}) \in \{-1, +1\}$$

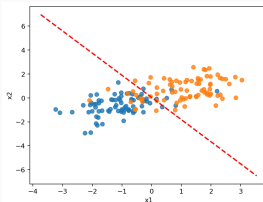
Decision boundary: $\mathbf{w}^\top \mathbf{x} + b = 0$

Linear Classification: Visualizations

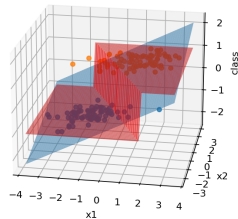
1D classification



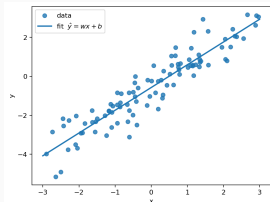
2D classification



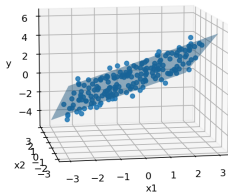
2D classification



1D regression



2D regression



Fit a linear model

Data and model:

$$\{(\mathbf{x}_i, y_i)\}_{i=1}^n, \quad \mathbf{x}_i \in \mathbb{R}^d, \quad y_i \in \mathbb{R} \text{ (regression)} \text{ or } \{-1, +1\} \text{ (classification)}$$

Prediction is $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$ or $\text{sign}(\mathbf{w}^\top \mathbf{x})$ - we just need to find $\mathbf{w}$:

Optimization (training): choose $\mathbf{w}$ to minimize average loss + regularization

$$\min_{\mathbf{w}} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(y_i, \mathbf{w}^\top \mathbf{x}_i) + \lambda \Omega(\mathbf{w})$$

Typical regularizers: $\Omega(\mathbf{w}) = \|\mathbf{w}\|_2^2$ (Ridge - L_2), $\Omega(\mathbf{w}) = \|\mathbf{w}\|_1$ (LASSO - L_1).

Loss choices:

- **Regression:** MSE, MAE
- **Classification:** SVM, Perceptron, LDA, Logistic (log-loss)

Loss choices for linear models

Let $z = \mathbf{w}^\top \mathbf{x}$.

Regression

- **MSE (squared error):**

$$\mathcal{L}_{\text{MSE}}(y, z) = (y - z)^2$$

Smooth, closed form, sensitive to outliers.

- **MAE (absolute error):**

$$\mathcal{L}_{\text{MAE}}(y, z) = |y - z|$$

Robust to outliers; solved via LP/IRLS (no closed form).

Classification (2 classes)

- **SVM** — max-margin with *hinge* loss
 $\mathcal{L}_{\text{hinge}}(y, z) = \max(0, 1 - yz)$
- **Perceptron** — perceptron loss
 $\mathcal{L}_{\text{perc}}(y, z) = \max(0, -yz)$
- **LDA** — Fisher criterion
- **Logistic (log-loss)**
 $\mathcal{L}_{\text{log}}(y, z) = \log(1 + e^{-yz})$

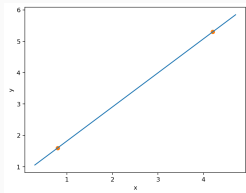
Linear regression with two points

Model (1D feature + intercept): $y = w x + b$

Equations to satisfy:
 $y_1 = w x_{11} + b$
 $y_2 = w x_{21} + b$

Matrix form:

$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & x_{11} \\ 1 & x_{21} \end{bmatrix}}_X \underbrace{\begin{bmatrix} b \\ w \end{bmatrix}}_{\tilde{w}} \iff \mathbf{y} = X \tilde{w}.$$



Inverse-form solution:

$$\tilde{w} = X^{-1} \mathbf{y},$$

Prediction for a new input x_{new} and more new samples in feature matrix X_{new} :

$$\hat{y}_{\text{new}} = w x_{\text{new}} + b, \quad \hat{\mathbf{y}}_{\text{new}} = X_{\text{new}} \tilde{w}$$

For two points, there exists a straight line that goes through both points. For more points, it is not possible $\rightarrow$ we need to make $\mathbf{y}$ and $X \tilde{w}$ as similar as possible.

MSE linear regression (Ordinary Least Squares - OLS)

We want predictions $X\mathbf{w}$ to be as close as possible to $\mathbf{y}$. We can define it with MSE (L_2 norm).

$$\underset{\mathbf{w}}{\text{minimize}} \quad \sum_{i=1}^n (\mathbf{x}_i^\top \mathbf{w} - y_i)^2 = \|X\mathbf{w} - \mathbf{y}\|_2^2$$

This is a simple convex linear optimization without constraints. We can employ various approaches (e.g., gradient descent), but a closed-form solution is also available:

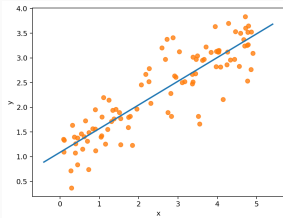
Objective (MSE): $g(\mathbf{w}) = \|X\mathbf{w} - \mathbf{y}\|_2^2 = (X\mathbf{w} - \mathbf{y})^\top (X\mathbf{w} - \mathbf{y})$

Derivative of cost: $\nabla g(\mathbf{w}) = 2X^\top (X\mathbf{w} - \mathbf{y})$

Set gradient to zero: $2X^\top (X\mathbf{w} - \mathbf{y}) = 0$

Closed-form solution: $\mathbf{w}^* = (X^\top X)^{-1} X^\top \mathbf{y} = X^\dagger \mathbf{y}$

Here $X^\dagger = (X^\top X)^{-1} X^\top$ denotes the Moore–Penrose **pseudoinverse**.



Linear regression with MAE (LAD - Least absolute deviations)

Objective (MAE):

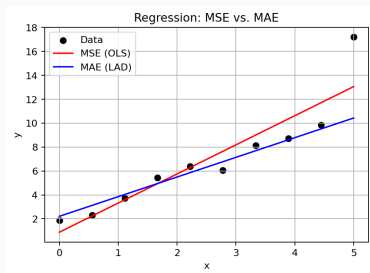
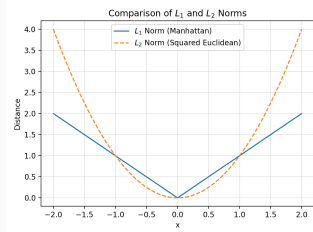
$$\underset{\mathbf{w}}{\text{minimize}} \quad \sum_{i=1}^n |\mathbf{x}_i^T \mathbf{w} - y_i| = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_1$$

No closed-form solution. Solved via optimization methods (e.g., LP).

$$\|x\|_2 = \|x\| = \sqrt{\sum_i x_i^2}$$

$$\|x\|_2^2 = x^T x = \sum_i x_i^2$$

$$\|x\|_1 = \sum_i |x_i|$$



L_1 is less sensitive to large values (outliers) and more sensitive to very small values (very small values are still penalized). $\rightarrow$ LAD is robust to outliers.
("Small values squared are very small and thus ignored - e.g., $0.01^2 = 0.0001$ ")

Ridge vs. LASSO regularization for linear regression

Ridge regression (L_2)

$$\underset{\mathbf{w}}{\text{minimize}} \quad \|X\mathbf{w} - \mathbf{y}\|_2^2 + \lambda \|\mathbf{w}\|_2^2$$

- Shrinks coefficients smoothly.
- Keeps all features (no exact zeros).
- Closed form:
 $\mathbf{w} = (X^\top X + \lambda I)^{-1} X^\top \mathbf{y}.$

LASSO regression (L_1)

$$\underset{\mathbf{w}}{\text{minimize}} \quad \|X\mathbf{w} - \mathbf{y}\|_2^2 + \lambda \|\mathbf{w}\|_1$$

- Performs feature selection (some $w_j = 0$).
- Robust to irrelevant features.
- No closed form, solved by convex optimization.

λ is hyper-parameter adjusting amount of regularization. (Weighing between fit and regularization term.) For LASSO - $\uparrow \lambda \rightarrow$ fewer features selected.

Same regularization can be used for other methods (LAD, linear classifiers) with similar properties.

Polynomial regression

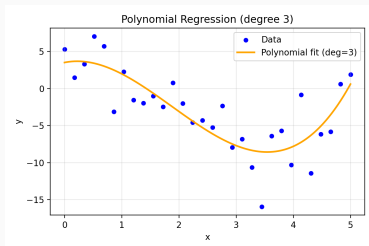
We can use the same approach as for linear regression to fit a non-linear function.

Example: m -th degree polynomial of 1D data

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_m x_i^m$$

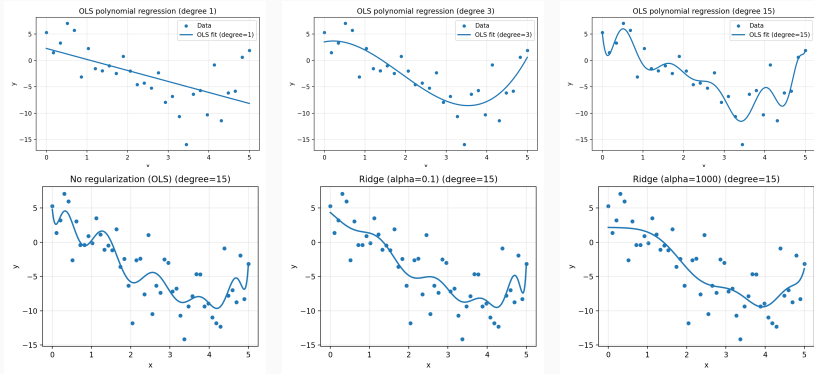
Which is equivalent to transforming features and then using linear regression:

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^m \\ 1 & x_2 & x_2^2 & \cdots & x_2^m \\ 1 & x_3 & x_3^2 & \cdots & x_3^m \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^m \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \vdots \\ \beta_m \end{bmatrix}$$



Estimation of coefficients is the same as in linear regression – only X is transformed. And similarly to PCA, we can do also implicit expansion using kernels (but we need dual form). [code example](#)

Effect of polynomial and regularization



We have 2 hyperparameters - degree of polynomial and amount of regularisation (λ).

Which one is example of over-fitting and which one is under-fitting?

Kernel Trick - from last lecture

Polynomial transform - degree 3 in 2D (dimensionality quickly grow with input dimension and degree)

$$\Phi^{(3)}(\mathbf{x}) = [1, x_1, x_2, x_1^2, x_1x_2, x_2^2, x_1^3, x_1^2x_2, x_1x_2^2, x_2^3]^\top$$

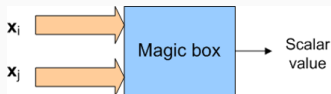
Problem: Explicit mapping $\Phi(\mathbf{x})$ can lead to very high (even infinite) dimensional feature spaces. Computing $\Phi(\mathbf{x})$ directly may be expensive or infeasible.

Key idea: Many ML algorithms, you can get *dual form*, where depend *only on inner products* $\Phi(\mathbf{x}_i)^\top \Phi(\mathbf{x}_j)$.

$$k(\mathbf{x}_i, \mathbf{x}_j) = \Phi(\mathbf{x}_i)^\top \Phi(\mathbf{x}_j)$$

If we can compute $k(\mathbf{x}_i, \mathbf{x}_j)$ **without computing $\Phi(\mathbf{x})$ explicitly**, we get the benefits of high-dimensional features at low cost.

Kernel Trick example - from last lecture



Two equivalent views:

1. Explicit feature expansion:

$$\Phi(\mathbf{x}) = (x_1^2, x_2^2, \sqrt{2} x_1 x_2)$$

$$\Phi(\mathbf{x}_i)^\top \Phi(\mathbf{x}_j) = x_{i1}^2 x_{j1}^2 + x_{i2}^2 x_{j2}^2 + 2x_{i1} x_{i2} x_{j1} x_{j2}$$

2. Direct computation (kernel trick):

$$(\mathbf{x}_i^\top \mathbf{x}_j)^2 = (x_{i1} x_{j1} + x_{i2} x_{j2})^2 = x_{i1}^2 x_{j1}^2 + x_{i2}^2 x_{j2}^2 + 2x_{i1} x_{i2} x_{j1} x_{j2}$$

Conclusion: Computing $\Phi(\mathbf{x}_i)^\top \Phi(\mathbf{x}_j)$ explicitly is the same as computing $(\mathbf{x}_i^\top \mathbf{x}_j)^2$. $\Rightarrow$ Kernel function avoids explicit high-dimensional mapping.

Kernel Function and Kernel Matrix - from last lecture

Kernel function: A function $k : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ such that

$$k(\mathbf{x}_i, \mathbf{x}_j) = \langle \Phi(\mathbf{x}_i), \Phi(\mathbf{x}_j) \rangle = \Phi(\mathbf{x}_i)^\top \Phi(\mathbf{x}_j)$$

for some (possibly high-dimensional) feature map $\Phi(\cdot)$.

Examples:

- Linear kernel: $k(\mathbf{x}_i, \mathbf{x}_j) = \gamma \mathbf{x}_i^\top \mathbf{x}_j + c \Rightarrow k(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^\top \mathbf{x}_j$
- Polynomial kernel: $k(\mathbf{x}_i, \mathbf{x}_j) = (\gamma \mathbf{x}_i^\top \mathbf{x}_j + c)^p \Rightarrow k(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^\top \mathbf{x}_j + 1)^p$
- RBF (Gaussian) kernel: $k(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma^2}\right) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$

Kernel matrix (Gram matrix): For n samples $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$,

$$K = \begin{bmatrix} k(\mathbf{x}_1, \mathbf{x}_1) & k(\mathbf{x}_1, \mathbf{x}_2) & \dots & k(\mathbf{x}_1, \mathbf{x}_n) \\ k(\mathbf{x}_2, \mathbf{x}_1) & k(\mathbf{x}_2, \mathbf{x}_2) & \dots & k(\mathbf{x}_2, \mathbf{x}_n) \\ \vdots & \vdots & \ddots & \vdots \\ k(\mathbf{x}_n, \mathbf{x}_1) & k(\mathbf{x}_n, \mathbf{x}_2) & \dots & k(\mathbf{x}_n, \mathbf{x}_n) \end{bmatrix}$$

Kernel function can “replace” $\mathbf{x}_i^\top \mathbf{x}_j$ and the kernel matrix can “replace” $XX^\top$.

Primal vs Dual Ridge regression

Primal form

$$\underset{\mathbf{w}}{\text{minimize}} \quad \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda \|\mathbf{w}\|_2^2$$

closed form solution:

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$$

prediction of a new sample:

$$\hat{y}_{\text{new}} = \mathbf{x}_{\text{new}}^\top \mathbf{w}$$

prediction for *many* new samples
(feature matrix $\mathbf{X}_{\text{new}} \in \mathbb{R}^{m \times d}$):

$$\hat{\mathbf{y}}_{\text{new}} = \mathbf{X}_{\text{new}} \mathbf{w}$$

Dual form

$$\underset{\boldsymbol{\alpha}}{\text{minimize}} \quad \frac{1}{2} \boldsymbol{\alpha}^\top (\mathbf{X}\mathbf{X}^\top + \lambda \mathbf{I}) \boldsymbol{\alpha} - \mathbf{y}^\top \boldsymbol{\alpha}$$

closed form solution:

$$\boldsymbol{\alpha}^* = (\mathbf{X}\mathbf{X}^\top + \lambda \mathbf{I})^{-1} \mathbf{y}$$

$$\mathbf{w} = \sum_{i=1}^N \alpha_i \mathbf{x}_i = \mathbf{X}^\top \boldsymbol{\alpha}$$

prediction of a new sample:

$$\hat{y}_{\text{new}} = \sum_{i=1}^N \alpha_i \mathbf{x}_i^\top \mathbf{x}_{\text{new}} = \boldsymbol{\alpha}^\top \mathbf{X} \mathbf{x}_{\text{new}}$$

prediction for *many* new samples:

$$\hat{\mathbf{y}}_{\text{new}} = \mathbf{X}_{\text{new}} \mathbf{X}^\top \boldsymbol{\alpha}$$

Again, primal form use $n \times n$ matrix $\mathbf{X}^\top \mathbf{X}$ and dual form use $d \times d$ $\mathbf{X}\mathbf{X}^\top$ matrix.
Which one is faster to use depend if you have more features or more samples.

Ridge regression: Dual & Kernel (KRR)

Dual form

$$\underset{\alpha}{\text{minimize}} \quad \frac{1}{2} \alpha^\top (\mathbf{X}\mathbf{X}^\top + \lambda \mathbf{I}) \alpha - \mathbf{y}^\top \alpha$$

closed form solution:

$$\alpha^* = (\mathbf{X}\mathbf{X}^\top + \lambda \mathbf{I})^{-1} \mathbf{y}$$

prediction of a new sample:

$$\hat{y}_{\text{new}} = \sum_{i=1}^N \alpha_i \mathbf{x}_i^\top \mathbf{x}_{\text{new}}$$

prediction for *many* new samples:

$$\hat{\mathbf{y}}_{\text{new}} = \mathbf{X}_{\text{new}} \mathbf{X}^\top \alpha$$

In dual form, all x are in form of dot product ($\mathbf{x}_i^\top \mathbf{x}_j$ or $\mathbf{X}\mathbf{X}^\top$), thus we can replace it with kernel function $k(\mathbf{x}_i, \mathbf{x}_j)$ and kernel matrix $\mathbf{K}$.

This implicit feature transformation gives same results and explicit one, but computational complexity does not change with the dimensionality. code example

Kernel Regression

$$\underset{\alpha}{\text{minimize}} \quad \frac{1}{2} \alpha^\top (\mathbf{K} + \lambda \mathbf{I}) \alpha - \mathbf{y}^\top \alpha$$

closed form solution:

$$\alpha^* = (\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{y}$$

prediction of a new sample:

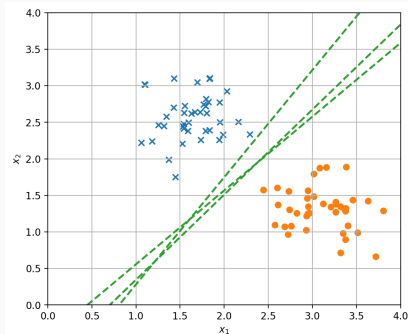
$$\hat{y}_{\text{new}} = \sum_{i=1}^N \alpha_i k(\mathbf{x}_i, \mathbf{x}_{\text{new}})$$

prediction for *many* new samples:

$$\hat{\mathbf{y}}_{\text{new}} = \mathbf{K}_{\text{new}} \alpha, \quad (\mathbf{K}_{\text{new}})_{ij} = k(\mathbf{x}_{\text{new},i}, \mathbf{x}_j)$$

Linearly separable data

Definition: Data are *linearly separable* if there exists a straight line (hyperplane in higher dimensions) that separates all positive from all negative samples without error.



Real data are noisy, overlapping, or mislabeled. $\Rightarrow$ Exact separability almost never holds. Theoretical concept for the concept of SVMs or perceptron convergence.

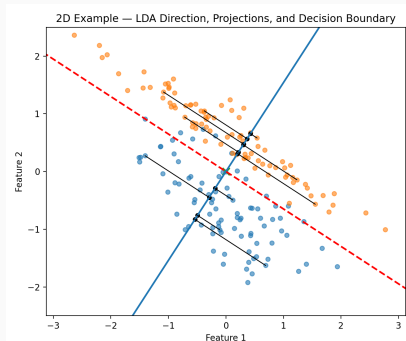
LDA - recap

Fisher criterion (maximize separation / penalize spread):

$$J(\mathbf{w}) = \frac{(\tilde{\mu}_1 - \tilde{\mu}_2)^2}{\tilde{s}_1^2 + \tilde{s}_2^2} = \frac{(\mathbf{w}^\top (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2))^2}{\mathbf{w}^\top S_W \mathbf{w}}, \quad S_W = S_1 + S_2.$$

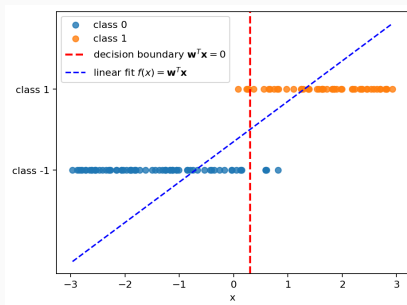
$$\mathbf{w} = S_W^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)$$

$$t = \frac{1}{2}(\boldsymbol{\mu}_1 + \boldsymbol{\mu}_2)^\top \mathbf{w} - \ln \frac{\pi_1}{\pi_2}$$



MSE classifier

You can train with MSE (same as regression) even with labels $y \in \{-1, 1\}$ and then use $\text{sign}(\cdot)$ to get a label from continuous prediction.



- simple, with closed form solution
- sensitive to outliers
- bad performance

Logistic regression

Idea: We should squeeze output values to range $[-1, 1]$ (or $[0, 1]$). E.g. with sigmoid:

Model (probabilistic):

$$p(y = 1 \mid \mathbf{x}) = \sigma(z), \quad z = \mathbf{w}^\top \mathbf{x} + b, \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

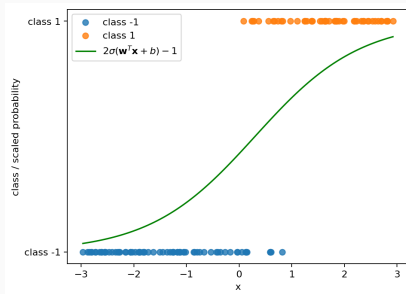
Decision rule:

$$\hat{y} = \begin{cases} 1 & \text{if } \sigma(z) \geq 0.5 \\ 0 & \text{otherwise} \end{cases} \quad \text{equivalently} \quad \hat{y} = \text{sign}(z) \quad \text{for } y \in \{-1, +1\}.$$

Loss (cross-entropy / logistic):

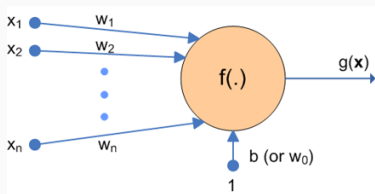
$$\mathcal{L}_{\log}(y, z) = \begin{cases} -y \log \sigma(z) - (1 - y) \log(1 - \sigma(z)) & \text{for } y \in \{0, 1\}, \\ \log(1 + e^{-yz}) & \text{for } y \in \{-1, +1\}. \end{cases}$$

Logistic regression



- Outputs calibrated probabilities (under correct model).
- Convex objective; solution e.g. gradient descent.
- Less sensitive to outliers than MSE.
- We can add regularization.
- This loss is used for deep learning (linear model replaced by neural network).

Perceptron algorithm



$$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x} + b), \quad y \in \{-1, +1\}.$$

$$\hat{y} = \text{sign}(\tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}) = \text{sign}\left(\sum_{j=0}^d w_j x_j\right)$$

$$\mathcal{L}_{\text{perc}}(y, \mathbf{w}; \mathbf{x}) = \max(0, -y \mathbf{w}^\top \tilde{\mathbf{x}})$$

Intuition: If a sample is misclassified ($y \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}} \leq 0$), we *shift* $\tilde{\mathbf{w}}$ toward $y \tilde{\mathbf{x}}$. Multiplication by $y \in \{-1, +1\}$ will automatically change sign depending on the sample label.

Simple iterative rule (learning rate $\eta > 0$):

$$\text{for } i = 1, \dots, n: \quad \text{if } y_i \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}_i \leq 0, \quad \tilde{\mathbf{w}} \leftarrow \tilde{\mathbf{w}} + \eta y_i \tilde{\mathbf{x}}_i.$$

This algorithm converges provably only for linearly separable classes.

Perceptron algorithm

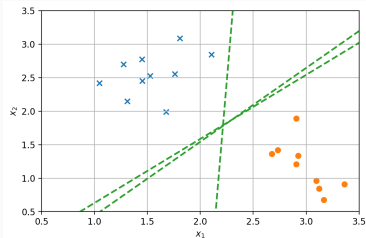
```
def train_perceptron(X, y, epochs):
    n, d = X.shape
    w = np.zeros(d)
    for _ in range(epochs):
        for i in range(n):
            y_hat = X[i] @ w
            if y[i] * y_hat <= 0:
                w += y[i] * X[i]
    return w

def predict_perceptron(X, w):
    return np.sign(X @ w)
```

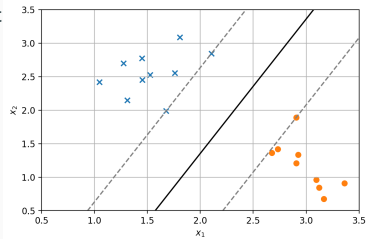
Key idea: Each update adds $y_i \mathbf{x}_i$ to $\mathbf{w} \Rightarrow$ Weights need to be weighted sum (linear combination) of samples: $\mathbf{w} = \sum_{i=1}^n \alpha_i \mathbf{x}_i$ Which is definition of weights α in dual form. (Intuitive "proof" of dual form.) code example

SVM intuition

For linearly separable data, which decision boundary to choose if all of them provide best classification?



Best one can be defined as one with largest **margin** ("largest safe space"):



Optimization problem: choose the separating hyperplane that maximizes the margin, i.e. the distance between the decision boundary and the closest points from each class. How to formulate this mathematically?

From intuition to SVM optimization

Step 1: Margin definition Distance of a point to hyperplane $(\mathbf{w}, b)$ is

$$\frac{|\mathbf{w}^\top \mathbf{x}_i + b|}{\|\mathbf{w}\|}$$

Step 2: Margin for correctly classified points For all i : $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 \Rightarrow$ closest points (support vectors) lie exactly at distance $1/\|\mathbf{w}\|$.

Step 3: Optimization Maximize the margin $\frac{1}{\|\mathbf{w}\|}$

$$\underset{\mathbf{w}, b}{\text{maximize}} \quad \frac{1}{\|\mathbf{w}\|} \quad \text{s.t. } y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$$

Equivalent standard form:

$$\underset{\mathbf{w}, b}{\text{minimize}} \quad \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.t. } y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1.$$

Intuition 1: The ≥ 1 condition just fixes the scale - we force the closest points to sit exactly at distance 1, so the margin is measured as $1/\|\mathbf{w}\|$ and is well defined.

Intuition 2: Instead of forcing $\|\mathbf{w}\| = 1$ and letting the margin vary, we fix the margin at 1 and let $\mathbf{w}$ scale adjust — so the margin becomes $1/\|\mathbf{w}\|$.

Hard-margin SVM with dual form

Optimization problem (primal):

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n$$

Geometric margin of closest points: $1/\|\mathbf{w}\|$ (margin band width = $2/\|\mathbf{w}\|$).

Dual (convex QP):

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j \quad \text{s.t.} \quad \sum_{i=1}^n \alpha_i y_i = 0, \quad \alpha \geq 0$$

$$\mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i, \quad \hat{y} = \text{sign}\left(\sum_{i \in \text{SV}} \alpha_i y_i \mathbf{x}_i^\top \mathbf{x} + b\right).$$

It is *quadratic programming* (QP) problem (in the dual) - we can use standard tools for QP - efficient algorithms with guaranteed global optima.

Support vectors (SV):

$\alpha_i > 0 \iff y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1$ (they lie on the margin and determine $\mathbf{w}, b$).

We need only SVs for prediction in dual ($\alpha_i y_i \mathbf{x}_i^\top \mathbf{x}$ is zero for $\alpha_i = 0$) $\rightarrow$ very fast prediction. Intuitively - only SVs are necessary to get same line. `code example`

Soft-margin SVM

$$\underset{\mathbf{w}, b}{\text{minimize}} \quad \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1. \quad \text{works only if linearly separable.}$$

Allow violations with penalization via slack variables $\xi_i \geq 0$:

$$\underset{\mathbf{w}, b, \xi}{\min} \quad \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

$C > 0$ trades off margin size vs. violations; $\xi_i = 0$: correctly classified with margin ≥ 1 ; $0 < \xi_i \leq 1$: correct but inside margin; $\xi_i > 1$: misclassified.

Eliminate ξ_i analytically: From constraints, the smallest feasible

$$\xi_i = \max(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)).$$

Plugging back gives the *unconstrained* problem:

$$\underset{\mathbf{w}, b}{\min} \quad \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \max(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)).$$

Soft-margin SVM (hinge loss form)

$$\min_{\mathbf{w}, b} \underbrace{\frac{1}{2} \|\mathbf{w}\|^2}_{\text{margin (regularization)}} + C \underbrace{\sum_{i=1}^n \max(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b))}_{\text{hinge loss}}$$

Interpretation:

- $\frac{1}{2} \|\mathbf{w}\|^2$: prefers large margin (smaller weights).
- Hinge loss: penalizes points *inside* the margin or misclassified.
- $C > 0$: trade-off—higher $C \Rightarrow$ fit training data harder, lower $C \Rightarrow$ wider margin / more tolerance.
- hinge loss can also be used for e.g. neural networks.

C-SVM vs. ν -SVM

C-SVM (standard soft-margin):

$$\min_{\mathbf{w}, b, \xi} \quad \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

- Hyperparameter $C \in [0, \infty)$ controls trade-off between margin width and training errors.
- Larger C : fewer violations, tighter fit, risk of overfitting.
- Smaller C : wider margin, more tolerance to errors, risk of underfitting.

ν -SVM (alternative parameterization):

$$\min_{\mathbf{w}, b, \xi, \rho} \quad \frac{1}{2} \|\mathbf{w}\|^2 - \nu \rho + \frac{1}{n} \sum_{i=1}^n \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq \rho - \xi_i, \quad \xi_i \geq 0$$

- Uses parameter $\nu \in (0, 1]$ instead of C .
- ν has an intuitive meaning:
 - Upper bound on fraction of margin errors (training errors).
 - Lower bound on fraction of support vectors.

Kernel SVM

Motivation: Soft-margin SVM in *dual form* depends only on dot products:

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \quad \text{s.t.} \quad \sum_i \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C.$$

Kernel trick: Replace the dot product with a kernel function

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j k(\mathbf{x}_i, \mathbf{x}_j) \quad \text{s.t.} \quad \sum_i \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C.$$

Decision function:

$$f(\mathbf{x}) = \text{sign} \left(\sum_{i \in \text{SV}} \alpha_i y_i k(\mathbf{x}_i, \mathbf{x}) + b \right).$$

Summary: Kernel SVM allows *nonlinear decision boundaries* in the input space while keeping the optimization problem the same size (depends on n , not feature dimension). Prediction depends on

SVM and Kernel SVM in Practice

Interpretation (linear SVM):

- $\mathbf{w}$: weights (importance) of features, $1/\|\mathbf{w}\| = \text{margin width}$.
- SVs: training points on or inside the margin; they alone define $\mathbf{w}, b$.

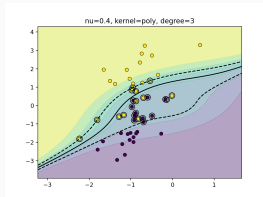
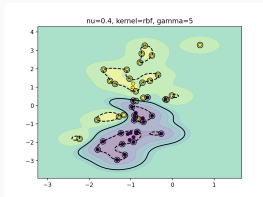
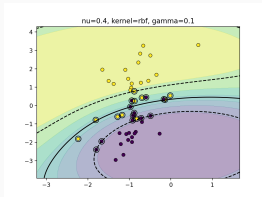
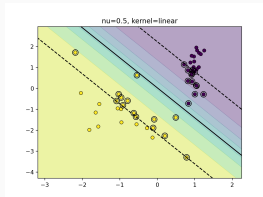
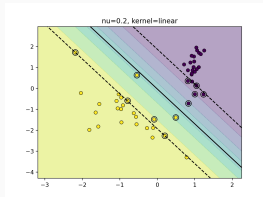
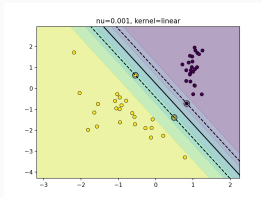
Interpretation (kernel SVM):

- No explicit $\mathbf{w}$ in input space.
- α_i : dual coefficients (nonzero only for SVs). Larger $\alpha_i = \text{more influence of that SV on the boundary}$.
 - Few SVs $\Rightarrow$ sparser, simpler model.
 - Many SVs $\Rightarrow$ more complex boundary, higher memory cost.

Hyperparameters to choose:

- C/ν — trade-off between margin width and training errors.
- Kernel type and kernel parameters:
 - Polynomial kernel: degree p - larger is more complex model (over-fitting).
 - RBF (Gaussian): width $\gamma = 1/2\sigma^2$ - larger is more complex model.

SVM visualizations



code example visualisation

code example application

Interpretation of $\mathbf{w}$ in Linear Models

Setup. For $z = \mathbf{w}^\top \mathbf{x} + b$: each coefficient w_j quantifies how changing feature x_j influences z (and the prediction), *holding other features fixed*.

General (all linear models).

$$\frac{\partial z}{\partial x_j} = w_j \quad \Rightarrow \quad \Delta z \approx w_j \Delta x_j$$

$\text{Sign}(w_j)$ indicates direction of effect; $|w_j|$ the strength.

With standardized features, larger $|w_j| \Rightarrow$ stronger predictive contribution.

Ridge shrinks all w_j ; LASSO yields sparsity (some $w_j = 0$) $\Rightarrow$ feature selection.

Collinearity & interpretation. If features are correlated, individual w_j may be unstable (trade-offs among collinear features). Prefer standardized inputs, regularization, and inspect groups of features.

Kernel models. In kernel SVM/KRR, $\mathbf{w}$ lives in feature space:

$\mathbf{w} = \sum_i \alpha_i \Phi(\mathbf{x}_i)$. No simple coordinate-wise interpretation in the original input space – $>$ more powerful model without interpretability.

Recap Quiz I — Linear Models Basics

- Explain the **bias trick**. How do we rewrite $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$ using $\tilde{\mathbf{x}}$ and $\tilde{\mathbf{w}}$? Why is this practical?
- What is the difference between **regression** and **classification** for linear models? State the role of the decision boundary $\mathbf{w}^\top \mathbf{x} + b = 0$.
- Why is **MSE** sensitive to outliers while **MAE** is more robust? Give the intuition.
- Write the generic **empirical risk minimization** form: average loss + regularization. What are typical choices of $\Omega(\mathbf{w})$ and their effects?
- Compare **logistic regression** with **MSE classifier**.
- In a linear classifier, what does a **larger margin** mean and how does it relate to $\|\mathbf{w}\|$?
- Give a scenario where you would prefer **MAE/LAD** over MSE. Briefly justify.

Recap Quiz II — OLS, Regularization, Polynomials

- Derive the **closed-form solution for OLS** from the objective $g(\mathbf{w}) = \|X\mathbf{w} - \mathbf{y}\|_2^2$.
- Contrast **Ridge** (L_2) vs. **LASSO** (L_1): coefficient shrinkage, feature selection, and the existence of a closed form.
- How does the **hyperparameter** λ affect the bias–variance trade-off? Give examples of *underfitting* and *overfitting*.
- How do you build **polynomial regression** via feature expansion? Explain why it remains linear in parameters.
- Using a simple example, explain how **polynomial degree** and **regularization** jointly shape the fit.

Recap Quiz III — Perceptron, SVM, Kernels

- Explain how the **perceptron** update implies $\mathbf{w}$ is a linear combination of training samples: $\mathbf{w} = \sum_i \alpha_i \mathbf{x}_i$.
- Define the **margin** in SVM and explain why maximizing the margin leads to minimizing $\frac{1}{2} \|\mathbf{w}\|^2$.
- Contrast **hard-margin** vs. **soft-margin** SVM. What is the role of C and what do the **slack** variables ξ_i represent?
- Interpret **support vectors** and **dual coefficients** α_i . What does *many* vs. *few* SVs mean for the model?
- Write the SVM **dual objective** and indicate where **dot products** (or kernels) appear between samples.
- Explain the **kernel trick**.
- Which **hyperparameters** do we tune in kernel SVMs (poly vs. RBF), and how do they affect decision boundary complexity?
- Why is the dual/kernel approach beneficial when the feature dimension d is very large or infinite?